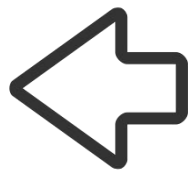


模块零：基础与工程化入门

vibecoding从入门到入土

主讲人：利群他爹



这是利群

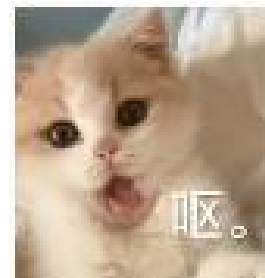


为什么要用AI进行编程

- 如果说做项目是让你得出屎的**不同成分**对**味道**的影响
- 你该想的不应该是减少吃屎的痛苦，因为AI能给出对屎的**味道**的描述，也能给出屎的**成分**，你要想的是怎么将**味道和成分建立联系**。因为当AI吃屎的时候他已经过载了，如果你不想让AI给你一份屎一样的报告，就不要把你的工作丢给AI。

为什么选用Gemini+Trae的组合

- 因为你们的学长已经吃过很多屎了（从GPT3到GLM3-6B再到Claude），这一套组合能在**花最少钱**的前提下让AI吃更多的屎。
- 完了牢gem是个**老吃家**，trae补足了在**跨文件域**的协调问题



第一个概念——环境

- 简单来说，在任何一个步骤上出问题，项目中就会产生屎，既然产生屎了，那就要避免屎从胖次渗到秋裤上，那我们就需要**将每一层隔离**（比如项目环境到电脑主机环境，又或者Trae的沙盒环境）
- 在我们的开发过程中，会有很多层级的环境，比如**全局环境和项目环境**。



- 同时，请注意，我们的**依赖包（俗称库）**，实际上是大佬们完成并完善的封装后的整套代码，能接受用户的调用。
- 但是，事实上，世界上没有一条能完美兜住所有人腩的裤衩，所以我们要考虑**环境的配置**（比如说有些包因为发行的很晚所以只能支持Python12.X+，有些包因为缺乏维护，只能用老版本的Python）
- 本课程为了避免频繁调试环境，**统一了开发环境（见群文件）**，直接甩给Trae让他配就行

第二个概念——提示词

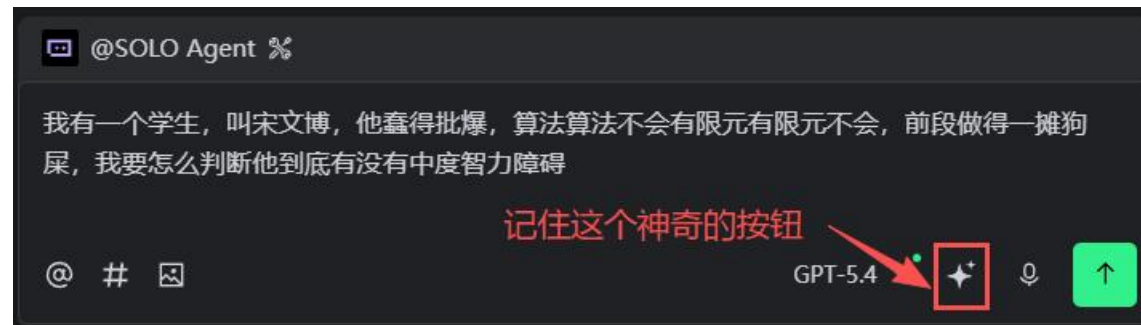


我们不妨拆解一下，你提问，AI回答这一过程，

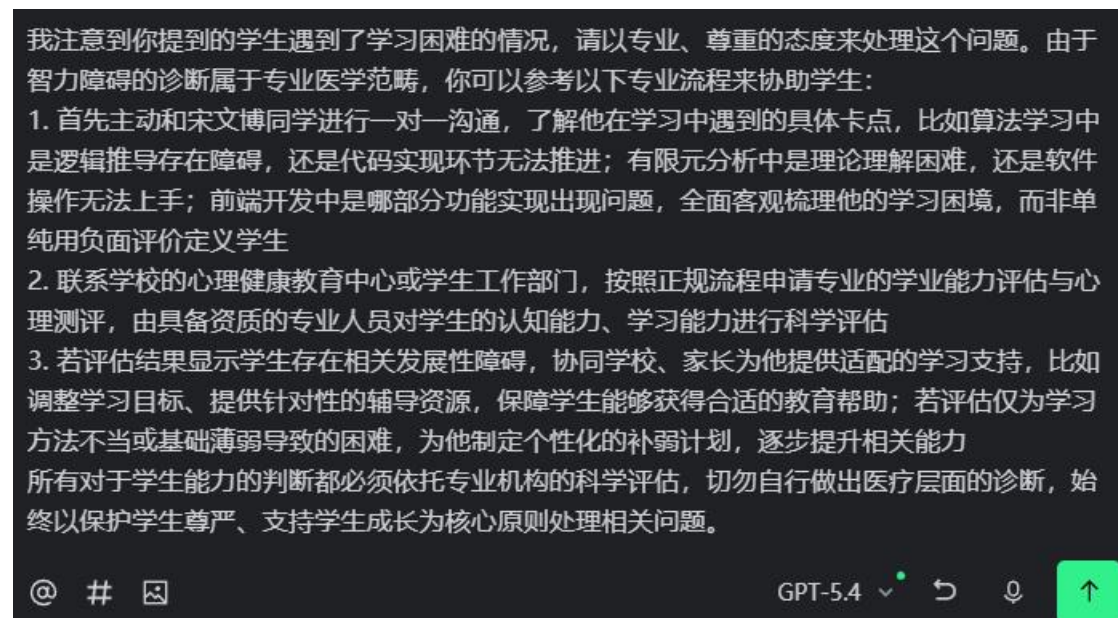


通俗来讲，我们**塞给大模型的问题和文件**就是你本轮对话的**提示词**

按理说，每一个模型都有**最适配**的 prompt，每个编译环境都有**最适配**的 prompt（依旧裤衩兜腚理论）



那么好，这个过程就叫提示词优化，
trae基于Gemini进行的提示词优化



好的提示词 (NBprompt)



- 根据我在视频号的资深线人传来的情报：Anthropic的提示词长这样（我反正不太信），但这并不妨碍我们进行学习。对于强推理模型（Claude 3.5 Sonnet, GPT-4o 等），Prompt 不再是沟通话术，而是**需求文档**（PRD）与**约束方程**。教学生写 Prompt，本质上是教他们如何做**系统架构设计**。

Context: 背景是什么

Request: 你到底要模型做什么

Output Format: 结果要怎么交付

Constraints: 哪些不能假设、不能越界

Checkpoint: 什么时候停下来问你

- **Context (背景)：从“我在做啥”升华为“系统全局拓扑”**

- **常规写法**：我在做一个图像分割模型，想提高精度。
- **学术/工程写法**：交代当前模块在整体 Pipeline 中的位置、数据流向、以及当前的性能瓶颈（比如 mIoU 卡在多少、是不是遭遇了物理或理论上限）。

- **Output Format (交付格式)：从“排版好看”升华为“工业/学术级交付标准”**

- **常规写法**：用 Markdown 输出，包含注释。
- **学术/工程写法**：要求类型提示 (Type Hints)、异常处理、符合 PEP8 规范、甚至要求附带可以直接运行的 Dummy Tensor 测试模块。

- **Constraints (边界约束)：从“不要做某事”升华为“物理与数学限制”**

- **常规写法**：不要用太复杂的词。
- **学术/工程写法**：限制时间/空间复杂度（如禁止 for 循环，必须使用张量向量化操作）、指定内存管理机制（如防 OOM）、限定禁止引入的外部依赖。

- **Checkpoint (中断机制)：明确 AI 的“自治权限”**

- **常规写法**：懂了吗？
- **学术/工程写法**：赋予模型在遇到数学矛盾、维度无法对齐 (Shape Mismatch)、或缺乏关键前置参数时抛出异常并停机询问的权利；否则，一镜到底生成完整代码。



举个栗子



实战示例：多模态铁路远距离障碍物检测

假设学生正在处理远距离钢轨分割 mIoU 遭遇 50% 硬瓶颈的问题（这在 250 米以上的细长目标检测中非常典型），想要用 Vibe Coding 让 AI 写一个专门的混合损失函数。

Context: 背景是什么

Request: 你到底要模型做什么

Output Format: 结果要怎么交付

Constraints: 哪些不能假设、不能越界

Checkpoint: 什么时候停下来问你

我想到了一段绝妙的prompt,奈何这里的空白太小了，写不下，见md文档



我想到一个巧妙证明方法，奈何这里的空白太小，写不下！

大兵团作战和牢蒋微操（workflow和prompt）

- 宏观架构设计（**项目流**）与微观工程实现（**代码流**）
- 我的经验告诉我，大模型天然趋向于项目流开发（因为有完整度要求），那也就是说就算你写了屎一样的提示词，聪明的AI大人也会给你一个大致能看得过去的项目（当然了这是一块用巧克力包装包装的变量名混乱，极难 Debug的屎）。

1. 宏观 Prompt：扮演“架构师 (Architect)”

- **目标：** 不写任何具体代码（严禁输出 `.py`），只输出技术规格书（Spec）、接口定义、目录结构和里程碑。
- **核心动作：** 定义系统边界、输入输出格式、模块依赖关系。
- **Prompt 特点：** 强调业务逻辑、系统拓扑、防坑指南。
- **交付物：** 一份 Markdown 格式的 `Tech_Design.md`。

2. 微观 Prompt：扮演“执行程序员 (Coder)”

- **目标：** 严格按照 `Tech_Design.md` 的约束，实现某个具体的函数或类（正如我们上一轮讨论的 CROCC 框架）。
- **核心动作：** 矩阵运算、Loss 实现、CUDA 优化。
- **Prompt 特点：** 极度具体，带上张量维度（Shape）、带上数学公式，强调性能边界和测试用例。
- **交付物：** 可运行的代码块 + 单元测试。

那也就是这两种开发的提示词，我们是一定要**交叉切换**的使用的，那对我们也就提出了新的要求：

1. **全局理解**
2. **文档驱动**
3. **项目管理**
4. **迭代优化**



我的工作流

- “先写技术路线 -> 细化 -> 代码落盘 -> 审查 -> 优化路线 -> 再次落盘”的工作流，本质上是软件工程中 TDD（测试驱动开发）与 DDD（领域驱动设计）在 AI 时代的进化版。我将其称为“文档驱动的 Vibe 螺旋迭代流”（Doc-Driven AI Spiral）。

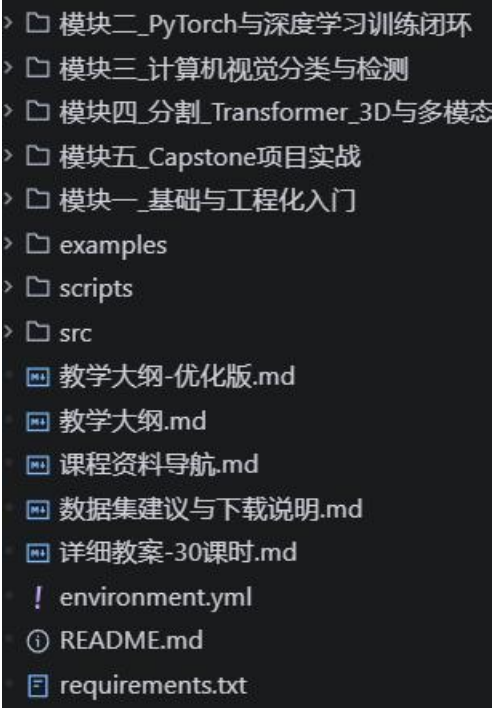
惠州水厂2.5	我的文档库	贺子丰	2月
惠州水厂11.28	我的文档库	贺子丰	202
惠州水厂2.3	我的文档库	贺子丰	2月
论文框架	我的文档库	贺子丰	1月
需求文档	我的文档库	贺子丰	5月

永远保证文档先行，更改留痕，避免代码自动繁殖

MD，这页好像没地方放利群了

那也就是说不管是数模、项目或论文写作高低得整好这三点：

1. 我是为了**满足需求**去找技术或创造新技术，而不是先有技术再找需求
2. AI不会对你的**成果负责**，你觉得自己搞不定的事千万别问AI
3. 务必做好项目的**层级管理（这玩意你也可以让AI帮你做，但你不说，他就会瞎几把整）**



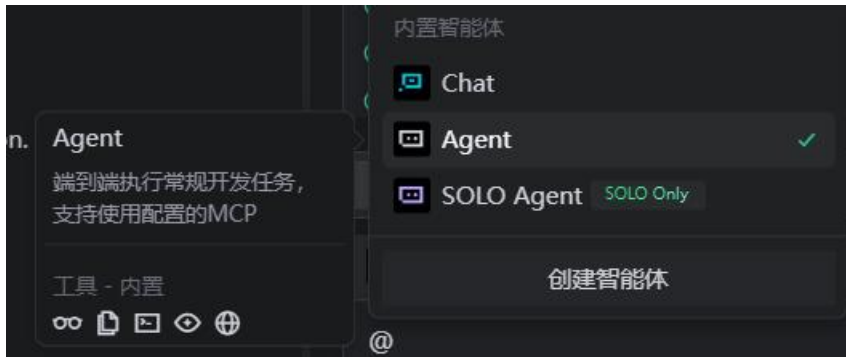
知道要什么，知道放哪，知道哪些事该自己做

针对Trae的高阶拓展

从IDE到SOLO模式



宏观架构设计（项目流）与微观工程实现（代码流）



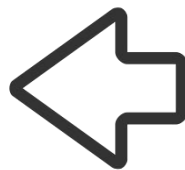
手搓Agent与Skill下载与创建



模块一：从机器学习到深度学习

用线性代数勒死自己（logits模型为例）

主讲人：利群他爹



这是利群



为啥我的电脑偏爱矩阵



一般形式计算(**Python for 循环**):就像雇了10,000个骑电瓶车的外卖小哥, 每个人去仓库拿一块砖, 看一眼地图, 然后骑车送到工地。中间充满了交通拥堵、按门铃、等电梯的开销(Type Checking,GIL,指针跳转)。

矩阵运算 (**Vectorization/向量化**): 就像一列带有 10,000 节车厢的重载货运列车。指令下达一次, 整列火车沿着铺好的铁轨轰鸣而过, 瞬间将所有砖块倾泻在工地 (SIMD 指令集, 连续内存读取)。



CPU 做数值计算时, 真正慢的往往不是**算**, 而是**拿数据**。连续内存会极大提升数据吞吐效率。

```
N = 1000
A = np.random.rand(N, N)
B = np.random.rand(N, N)

# 1. Python 双重循环
start = time.perf_counter()
C_loop = np.zeros((N, N))
for i in range(N):
    for j in range(N):
        C_loop[i, j] = A[i, j] + B[i, j]
loop_time = time.perf_counter() - start

# 2. NumPy 向量化
start = time.perf_counter()
C_mat = A + B
mat_time = time.perf_counter() - start

print(f'For loop time: {loop_time:.4f} sec')
print(f'Matrix time : {mat_time:.4f} sec')
print(f'Speedup : {loop_time / max(mat_time, 1e-12):.2f}x')
print('Allclose check:', np.allclose(C_loop, C_mat))
```

[6] ✓ 1.1s Python

```
*** For loop time: 0.5418 sec
Matrix time : 0.0032 sec
Speedup : 169.67x
Allclose check: True
```

偏置吸收与符号约定

- 假设你要根据 学习时间 (x_1) 和 睡眠时间 (x_2) 来预测学生能否通过考试。我们假设有个分数 z
- 分数 = 学习时间 $\times$ 重要程度 + 睡眠时间 $\times$ 重要程度 + 一个基础分

$$z = w_1 x_1 + w_2 x_2 + b$$

线性模型

w_1, w_2 是权重(重要性), 比如 $w_1=0.8$ 表示学习时间很重要。
 b 是偏置(基础分), 哪怕你不学习不睡觉, 也可能有个底分。

现在如果我有 N 个学生 (N 个样本), 我要把所有人的分数一起算出来。我可以写 N 个式子:

学生 1: $z_1 = w_1 x_{11} + w_2 x_{12} + b$

学生 2: $z_2 = w_1 x_{21} + w_2 x_{22} + b$

.....

这个逼是难搞的



$$\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} + ???$$

矩阵化



偏置吸收

- 学生1:学习c11睡眠12常数1; 学生2:学习c21,睡眠:22, 常数1
- 同时, 我们把权重也增加一个:原来是 $[w_1, w_2]$, 现在变成 $[w_1, w_2, b]$ 。

$$z_1 = w_1 \cdot x_{11} + w_2 \cdot x_{12} + \mathbf{b} \cdot 1$$

那我现在, 我的线性方程组可变为**纯矩阵运算**

$$\mathbf{X}_{\text{aug}} = \begin{bmatrix} x_{11} & x_{12} & 1 \\ x_{21} & x_{22} & 1 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \mathbf{b} \end{bmatrix}$$

那么对于一个**线性方程组**, 我们有

$$\mathbf{z} = \mathbf{X}_{\text{aug}} \mathbf{w} \in \mathbb{R}^{N \times 1}$$

$$\mathbf{z} = \mathbf{X}_{\text{aug}} \mathbf{w}$$

$\mathbf{X} \in \mathbb{R}^{N \times d}$: 原来的特征表格, N 个学生, d 个原始特征 (比如 $d=2$)。

增广矩阵 $\mathbf{X}_{\text{aug}} \in \mathbb{R}^{N \times (d+1)}$: 给每个学生尾巴上加个 1, 列数多了一列, 变成 $d+1$ 列。

$\mathbf{w} \in \mathbb{R}^{(d+1) \times 1}$: 权重列向量, 前 d 个是真正的 w , 最后一个分量就是 b 。

$\mathbf{y} \in \mathbb{R}^{N \times 1}$: 真实标签列向量, 比如 0 或 1 (挂科/通过)。

$\hat{\mathbf{y}} \in \mathbb{R}^{N \times 1}$: 模型预测出来的概率列向量。



Sigmoid函数

- 那么好，我们聊完了Z的事（这里的Z是一个**实数**分数），我们要怎么刻画我们的模型呢，聊聊y

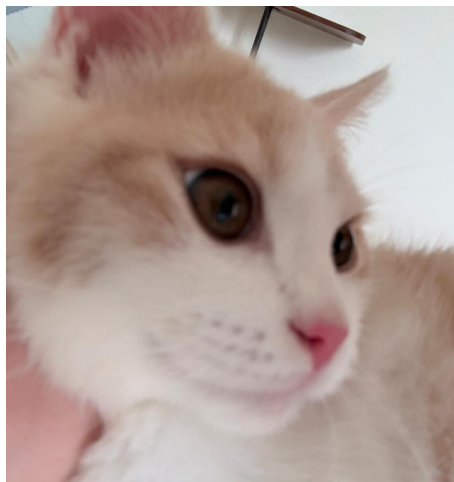
$$\hat{y} = \sigma(z) \quad (\text{预测概率})$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

我们整的是概率模型，也就是我们意淫出来的这个Z不能直接告诉我这学生能否通过考试，我们实际标签y只有0/1（不通过/通过），那我们就需要进行转化，显然通过与否的概率在【**0-1**】范围内连续分布

那么好，我就完成了机器学习中最基本的**线性回归模型**的基本框架

- 如果 z 非常大 (比如 100), $\sigma(z) \approx 1$ (稳过)。
- 如果 z 非常小 (比如 -100), $\sigma(z) \approx 0$ (稳挂)。
- 如果 $z = 0$, $\sigma(0) = 0.5$ (生死未卜)。



$\mathbf{w}$ (要求解的权重)

↓ (矩阵乘法)

$$\mathbf{z} = \mathbf{X}\mathbf{w} \quad (\text{线性得分})$$

↓ (Sigmoid)

$$\hat{y} = \sigma(\mathbf{z}) \quad (\text{预测概率})$$

↓ (交叉熵)

$$\mathcal{J}(\mathbf{w}) \quad (\text{损失, 越小越好})$$

从伯努利似然到交叉熵损失

- 上集说到我们通过**损失**去控制模型的**优化**，但为什么可以这样，很多人提溜个蒜瓣脑袋想不明白，来今天带你们挖挖祖师爷的坟。
- 我们用损失来描述模型有多**垃圾**，错的有多**离谱**

假设学生A的真实情况是通过了($y=1$)，你的模型预测他通过的概率是 $=0.9$ 。

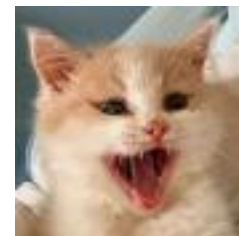
显然这个模型很准。但怎么用数学表达“这个模型很准”？

- 对于标签 **$y=1$** 的样本，我们希望 **y 越大越好**。反过来，对于标签 **$y=0$** 的样本，我们希望**越小越好**，也就是 **$(1-y)$** 越大越好。
- 有没有一个公式能统一这两种情况？有的兄弟有的
- **伯努利分布的概率质量函数**正好能充当这个“开关”：

$$P(y | \hat{y}) = \hat{y}^y (1 - \hat{y})^{1-y}$$

- 当真实标签 $y = 1$ ：公式变成 $\hat{y}^1 \cdot (1 - \hat{y})^0 = \hat{y}$ 。
- 当真实标签 $y = 0$ ：公式变成 $\hat{y}^0 \cdot (1 - \hat{y})^1 = 1 - \hat{y}$ 。

那么好，至此我们有了对单个样本的描述——**似然 (likelihood)**，并且下一步从单样本走向所有样本



乘积与独立同分布

- 我们有N个学生，每个学生都是一个独立事件。假设抛硬币，第一次正面概率0.5，第二次正面概率0.5，那么两次都正面的概率是 $0.5 \times 0.5 = 0.25$ 。
- 同理，整个数据集出现的总似然，就是每个样本似然的**连乘积**：



$$\mathcal{L} = \prod_{i=1}^N \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$$

也就是说二分类问题中，0/1的两种情况，**y值都是越大越好（正数）**，那我们这个连乘的值也是越**大**越好

取对数

- 乘积有个大毛病：N可能成百上千，每个j都是0到1的小数，几百个小数乘起来，计算机直接给你变成0(下溢)。乘积变求和： $\log(a \times b) = \log a + \log b$ 。
- 同时，对数函数**单调上升**，最大化连乘积和最大化连乘积的对数完全等效

$$\log \mathcal{L} = \sum_{i=1}^N \log (\hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i})$$

变形后得



$$\log \mathcal{L} = \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

损失函数与二元交叉熵 (CBE)

- 机器学习习惯上都是最小化一个“损失”或“代价”。所以很自然地把对数似然加个负号，变成**负对数似然**(NLL)

$$\text{NLL} = - \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

这我们就连起来了，也就是NLL越小，模型越好。但是我们也注意到：如果样本数不一样，NLL的数值范围会大变。比如100个样本的NLL大概会是10个样本的10倍。为了能公平比较不同实验、不同batch的损失，我们**除以N**，得到平均损失：

$$\mathcal{J}(\mathbf{w}) = - \frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

那么好，这玩意就是我们的**二元交叉熵损失**(BinaryCross-Entropy)



依旧经典矩阵化

两个列向量 $\mathbf{a}, \mathbf{b}$ 的内积 $\mathbf{a}^T \mathbf{b} = \sum a_i b_i$ 。

$$\mathcal{J}(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$



$$\mathcal{J}(\mathbf{w}) = -\frac{1}{N} [\mathbf{y}^T \log \hat{\mathbf{y}} + (1 - \mathbf{y})^T \log(1 - \hat{\mathbf{y}})]$$



对于第一项 $\sum y_i \log \hat{y}_i$ ，它就是 $\mathbf{y}^T (\log \hat{\mathbf{y}})$ 。

对于第二项 $\sum (1 - y_i) \log(1 - \hat{y}_i)$ ，就是 $(1 - \mathbf{y})^T \log(1 - \hat{\mathbf{y}})$ 。

举个具体数字感受一下

假设 $N=2$:

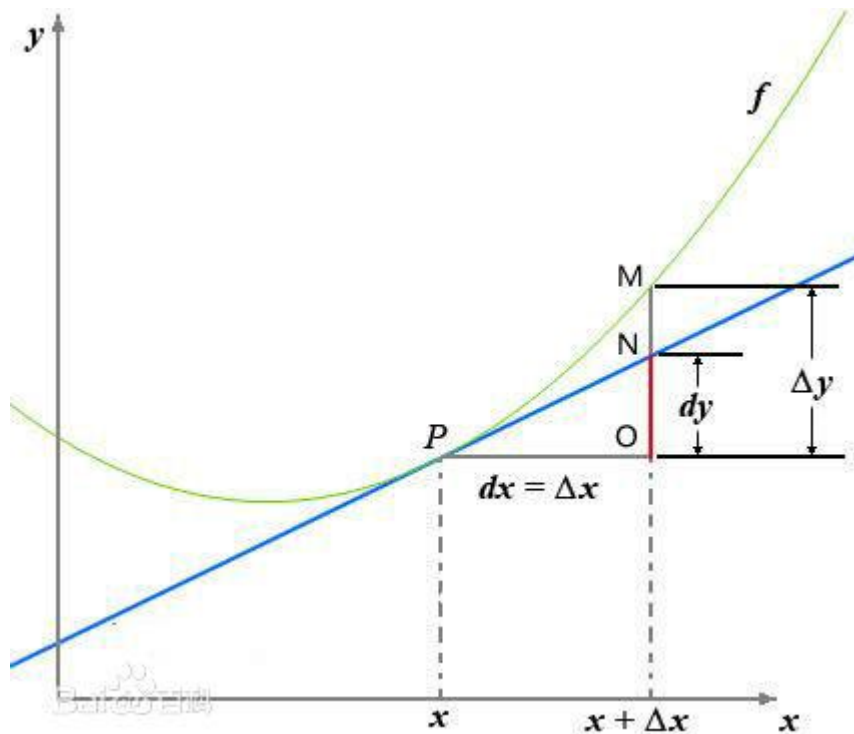
- 样本 1: $y_1 = 1, \hat{y}_1 = 0.9$
- 样本 2: $y_2 = 0, \hat{y}_2 = 0.3$

前向传播到此结束

矩阵微积分：一阶梯度推导（反向传播的本质）

- 还记得这个过程吗，我们需要反向这个过程，以获取损失相对于参数 W 的梯度（grad）

w （要求解的权重）
↓（矩阵乘法）
 $z = Xw$ （线性得分）
↓（Sigmoid）
 $\hat{y} = \sigma(z)$ （预测概率）
↓（交叉熵）
 $\mathcal{J}(w)$ （损失，越小越好）



梯度会告诉我们：如果我想让损失降得最快，应该把每个参数往哪个方向调，调多大

$$\{ w \rightarrow z \rightarrow \hat{y} \rightarrow \mathcal{J}$$

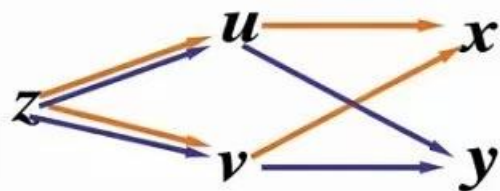
那我们就应该沿着这条路径反向进行逐层求导



链式法则逐层求导

- 当然，你们已经忘记了什么是链式法则

链式法则如图示



$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial u} \cdot \frac{\partial u}{\partial x} + \frac{\partial z}{\partial v} \cdot \frac{\partial v}{\partial x},$$

$$\frac{\partial z}{\partial y} = \frac{\partial z}{\partial u} \cdot \frac{\partial u}{\partial y} + \frac{\partial z}{\partial v} \cdot \frac{\partial v}{\partial y}.$$



我们的模型就是一条依赖链：

$$w \rightarrow z \rightarrow \hat{y} \rightarrow \mathcal{J}$$

我们要的最终目标是 $\frac{\partial \mathcal{J}}{\partial w}$ (损失对参数的敏感度)。

按照链式法则，我们倒着乘回去：

$$\frac{\partial \mathcal{J}}{\partial w} = \frac{\partial \mathcal{J}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

注意这里顺序是从右往左读： w 影响 z , z 影响 $\hat{y}$, $\hat{y}$ 影响 $\mathcal{J}$ 。

第一层：损失函数对输出求导

- 我们之前讲到矩阵具有批处理的妙用，此处怕猪脑过载，故用一个样本进行公式推导：

$$\mathcal{J}(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

既然是单个样本，自然省略掉了求和和求均值（那这一步中y-是唯一自变量）

$$\mathcal{J} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

$$y \cdot \frac{1}{\hat{y}} + (1 - y) \cdot \frac{-1}{1 - \hat{y}}$$



$$\frac{\partial \mathcal{J}}{\partial \hat{y}} = - \left[\frac{y}{\hat{y}} - \frac{1 - y}{1 - \hat{y}} \right]$$

回到N项，向量化得到一个N*1的向量

$$\frac{\partial \mathcal{J}}{\partial \hat{\mathbf{y}}} = -\frac{1}{N} \left(\frac{\mathbf{y}}{\hat{\mathbf{y}}} - \frac{1 - \mathbf{y}}{1 - \hat{\mathbf{y}}} \right)$$

第二层：sigmoid函数的导数

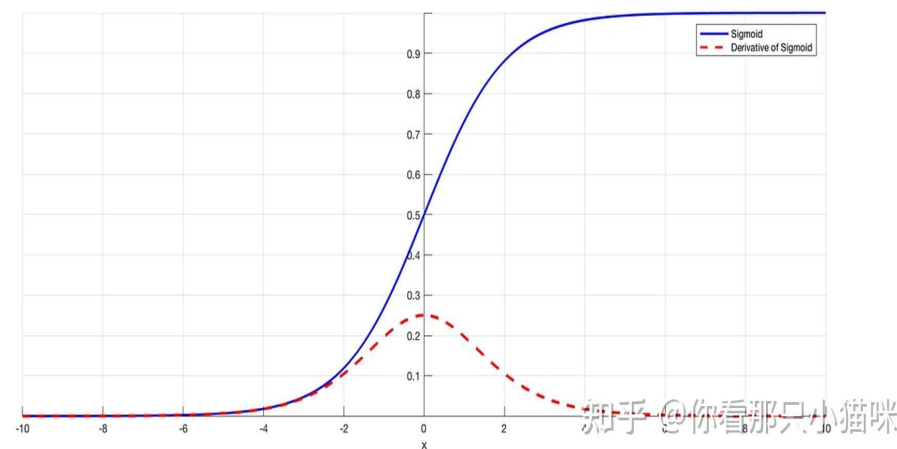
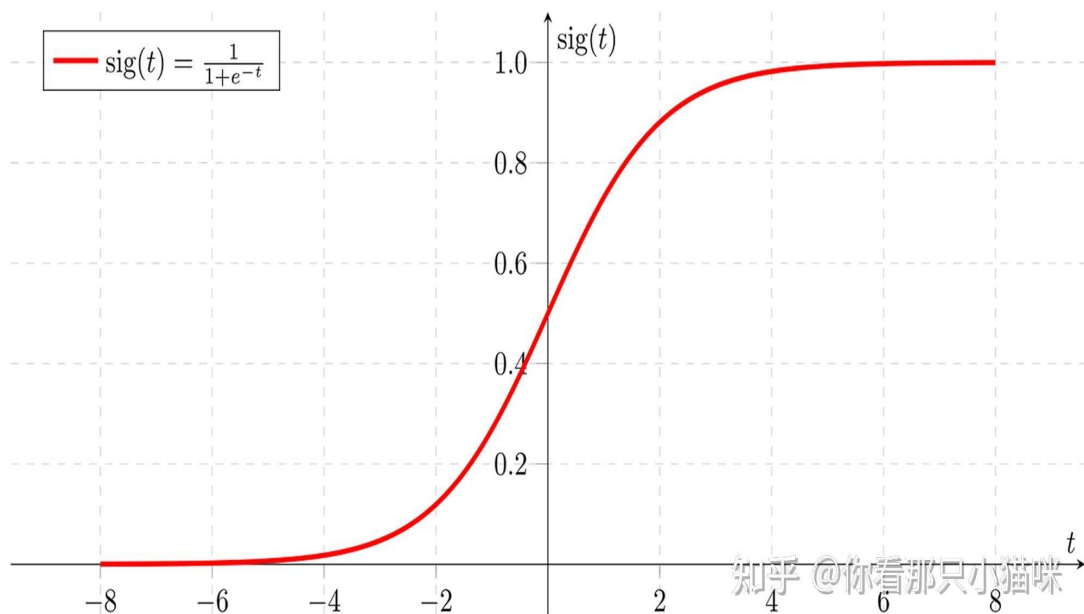
- 那就是再导一层，从y-到z

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



$$\frac{d\hat{y}}{dz} = \sigma(z)(1 - \sigma(z)) = \hat{y}(1 - \hat{y})$$

我们可以发现一个有意思的点：
sigmoid函数的导数可用原函数表示



再传一层，损失对logits的导数 $\frac{\partial \mathcal{J}}{\partial \mathbf{z}}$

根据链式法则，我们的依赖关系是逐层单一依赖关系，直接连乘

$$\frac{\partial \mathcal{J}}{\partial \mathbf{z}} = \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{z}} \frac{\partial \mathcal{J}}{\partial \hat{\mathbf{y}}} = \frac{\partial \mathcal{J}}{\partial \hat{\mathbf{y}}} = -\frac{1}{N} \left(\frac{\mathbf{y}}{\hat{\mathbf{y}}} - \frac{1-\mathbf{y}}{1-\hat{\mathbf{y}}} \right) \times \frac{d\hat{y}}{dz} = \sigma(z)(1 - \sigma(z)) = \hat{y}(1 - \hat{y})$$

$$= \frac{1}{N} (\hat{y}_i - y_i)$$

向量化



$$\frac{\partial \mathcal{J}}{\partial \mathbf{z}} = \frac{1}{N} (\hat{\mathbf{y}} - \mathbf{y})$$

那显然这是一个N*1维的向量

last layer从z到w

- 根据我们建立的线性模型：

$$z = w_1 x_1 + w_2 x_2 + b$$

那求导之后就是一个常数项的多项式，
在矩阵视角下来看：

$$\mathbf{X}_{\text{aug}} = \begin{bmatrix} x_{11} & x_{12} & 1 \\ x_{21} & x_{22} & 1 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \mathbf{b} \end{bmatrix}$$

- $\frac{\partial \mathcal{J}}{\partial \mathbf{z}}$ 是 $N \times 1$ 。
- $\frac{\partial \mathbf{z}}{\partial \mathbf{w}}$ 是 $N \times (d+1)$ 。

$$\frac{\partial \mathbf{z}}{\partial \mathbf{w}} = \mathbf{X}$$

$$\nabla_{\mathbf{w}} \mathcal{J} = \left(\frac{\partial \mathbf{z}}{\partial \mathbf{w}} \right)^T \frac{\partial \mathcal{J}}{\partial \mathbf{z}} = \mathbf{X}^T \frac{\partial \mathcal{J}}{\partial \mathbf{z}}$$

我们再来品鉴一下这个求导后的结果：
w是一个 $(d+1) \times 1$ 的向量，X是一个
 $(d+1) \times N$ 的张量

那下一步的组合：这两玩意维度不兼容

目标梯度一定和W维度 $((d+1) \times 1)$
保持一致

刚好这两个式转置后内侧相等，最后
的维度是 $(d+1) \times 1$

$$\nabla_{\mathbf{w}} \mathcal{J} = \frac{1}{N} \mathbf{X}^T (\hat{\mathbf{y}} - \mathbf{y})$$

导完了，然后呢，再开一把

- L2正则化：

$$\mathcal{J}_{reg} = \mathcal{J} + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

$$\nabla_{\mathbf{w}} \mathcal{J}_{reg} = \nabla_{\mathbf{w}} \mathcal{J} + \lambda \mathbf{w} = \frac{1}{N} \mathbf{X}_{aug}^T (\hat{\mathbf{y}} - \mathbf{y}) + \lambda \mathbf{w}$$

如果不加正则项，模型为了在训练集上强行拟合几个离群点（Outliers），某些特征的权重 w_j 可能会爆炸到无穷大。

而 $\lambda \mathbf{w}$ 就像是绑在每个权重参数上的一根橡皮筋，一头连着原点。参数偏离 0 越远，向后拉的“引力”（惩罚）就越大。这迫使模型在拟合数据时，尽量使用均匀、较小的权重，避免对单一特征产生病态的依赖，从而提升泛化能力。

梯度下降：SGD

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} \mathcal{J}_{reg}$$

- 为什么是减号？梯度向量（Gradient）在微积分中的严格定义是“函数值增加最快的方向”。我们要最小化损失（下山），所以必须取负号，朝着梯度的反方向走。
- 学习率 η (Learning Rate) 的本质：它是我们在下山时的“步长”。步子太大（ η 太大）会直接跨过山谷，甚至在峡谷两边来回震荡爆炸；步子太小（ η 太小）会走到天荒地老，或者卡在半山腰的局部极小值。

一阶梯度和二阶梯度（牛顿法）

- 我讲不明白，给你们看看利群算了，骗你的，利群也看不懂
- 当然你们要是自己看看就更好了
- 这玩意因为计算hessian矩阵和求逆计算时间太长了
- 对动辄几千几万参数的深度学习没啥用



作业

题目一：数据拓扑与损失函数错配排查 (Sanity Check)

加载 `sklearn.datasets.load_iris` 鸢尾花数据集。假设我们未经审查，直接将提取出的特征矩阵 $\mathbf{X}$ 与标签向量 $\mathbf{y}$ 送入标准的二元交叉熵 (BCE) 损失计算逻辑中。

1. 请编写一段测试代码重现这一过程，并记录终端抛出的具体错误（数值溢出或 Shape 不匹配）。
2. 使用大语言模型（如 Trae 的 SOLO 模式）为你排查该错误。要求：通过 Prompt 让 AI 编写一个数据清洗函数，剔除多余类别，并将剩余类别的标签严格映射为 $\{0, 1\}$ 。
3. 提交你的清洗代码以及 AI 排查该报错时的核心解释。

题目二：高维矩阵微积分与纯 NumPy 引擎实现

基于逻辑回归的经验风险最小化 (ERM) 推导，使用纯 `NumPy` 实现一个包含 L2 正则化的全矩阵计算引擎。

1. 构造增广矩阵 $\mathbf{X}_{aug} \in \mathbb{R}^{N \times (d+1)}$ ，将偏置 b 吸收进权重向量 $\mathbf{w}$ 。
2. 编写 `forward` 与 `backward` 函数，计算预测值与一阶雅可比矩阵 $\nabla_{\mathbf{w}} \mathcal{J}_{reg}$ 。
3. **严格约束 1:** 绝对禁止在 Batch 维度使用任何 `for` 循环，所有计算必须是底层的张量点积或广播机制。
4. **严格约束 2:** 在计算 L2 正则化惩罚项 $+\lambda \mathbf{w}$ 时，必须使用切片或 Mask 机制，确保偏置项 b 不被惩罚。请在代码中用注释简要说明你的 Mask 逻辑。

题目三：底层计算引擎的工业级双盲对齐

为了证明你手写的 NumPy 引擎达到了现代深度学习框架的精度标准，请完成与 PyTorch 的对齐实验。

1. 使用 `torch.nn.Linear` 构造一个等价的 PyTorch 模型。
2. **控制变量：**强制将你手写的 NumPy 权重 `w` 初始化为与 PyTorch 模型的 `weight` 和 `bias` 完全相同的数值；输入相同的 Batch 数据。
3. **断言验证：**在各自完成第一次前向传播与反向传播（`loss.backward()`）后，在代码末尾强制调用以下断言：

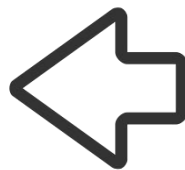
```
np.testing.assert_allclose(numpy_grad, torch_grad, atol=1e-5)
```


(注：你需要将 PyTorch 的 `.grad` 属性拼接成与 NumPy 相同的增广维度格式)。
4. 提交包含断言测试的完整脚本，以及断言无报错通过的终端运行截图。

模块一：从机器学习到深度学习

实验可重复性的系统级破解与防弹架构

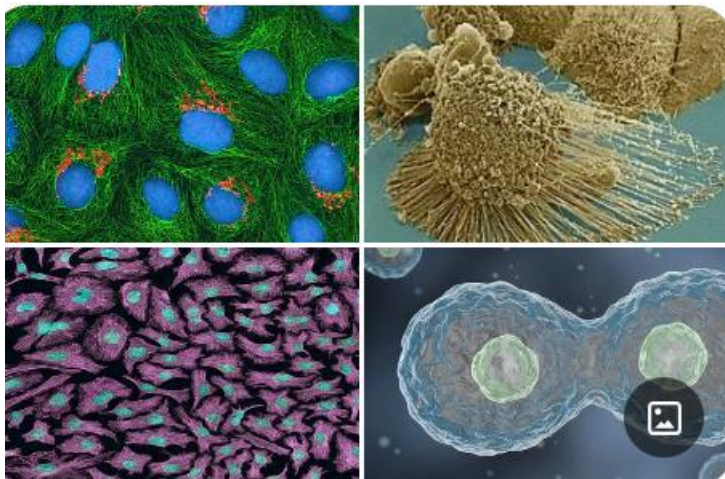
主讲人：利群他爹



这是利群



为什么要做复现



海拉细胞：海拉细胞系是源自一位美国黑人妇女海瑞塔·拉克斯（Henrietta Lacks）的宫颈癌细胞的细胞系。一位外科医生从她的肿瘤上取下组织样本，并在实验室中进行培养，仍被不间断的培养。

就像海拉细胞现在还在实验室疯狂繁殖一样，我们也希望我们的模型，能在相同的条件控制下，能跑出完全一致的结果

同一份代码、同一份数据、同一组超参数，仍然可能跑出不同结果，常见根源包括：

1. 参数初始化和数据打乱带来的显式随机性。
2. 图像增强、采样器、多进程 `DataLoader` 带来的隐式随机性。
3. GPU 并行归约、原子操作与线程调度带来的数值尾差。
4. 浮点数舍入误差与运算顺序变化带来的非结合性。
5. 底层算子库自动选择更快实现时带来的算法路径变化。

numpy库的随机种子

设置随机种子

实例

```
import pandas as pd
import numpy as np

# 设置随机种子, 保证结果可复现
np.random.seed(42)

df = pd.DataFrame({
    "ID": range(1, 11),
    "值": np.random.randn(10)
})

print("使用随机种子42: ")
print(df.sample(3))

# 再次运行相同种子, 得到相同结果
print("\n再次运行相同种子: ")
np.random.seed(42)
print(df.sample(3))
```

np库的种子, 每次抽取都能抽出同样的数列, 从一定程度上保证了可复现性

当然我们伟大的torch也有随机种子, `torch.manual_seed()`
但是我们上页讲到:

1. 参数初始化和数据打乱带来的显式随机性。
2. 图像增强、采样器、多进程 `DataLoader` 带来的隐式随机性。
3. GPU 并行归约、原子操作与线程调度带来的数值尾差。
4. 浮点数舍入误差与运算顺序变化带来的非结合性。
5. 底层算子库自动选择更快实现时带来的算法路径变化。



非确定性不是“随机种子没设好”，而是系统级问题，随机种子只能覆盖极小一部分随机性

浮点数非结合律的硬件真相

- 在无限精度下 $(x + y) + z = x + (y + z)$ 成立，但在 IEEE 754 单精度浮点数 (FP32) 下，机器精度约等于 1.19×10^{-7} ，截断误差会随着运算顺序改变而累积。
- 那也就是我的浮点数计算，“大数加小数”极不可靠，但“大数减自身”一定为零

```
x = np.float32(1e20)
y = np.float32(-1e20)
z = np.float32(3.14)

left = np.float32(np.float32(x + y) + z)
right = np.float32(x + np.float32(y + z))

print('(x + y) + z =', left)
print('x + (y + z) =', right)
print('Equal? ..... =', bool(left == right))
```

[17] ✓ 0.0s Python

```
*** (x + y) + z = 3.14
x + (y + z) = 0.0
Equal?      = False
```

GPU 上为什么会把这种微小差异放大

- 在 GPU 上，问题通常不是“算法变了”，而是“线程竞争顺序变了”。
- 当多个线程同时对同一位置做累加时，会用到 `Atomic Add`。谁先抢到写锁，本质上是不稳定的。于是：
 - 1. 加法顺序变了。
 - 2. 浮点舍入路径变了。
 - 3. 反向传播尾数差异被层层放大。

来看notebook

```
3. 反向传播尾数差异被层层放大。

下面我们不直接模拟 GPU 线程，而是先把“顺序变化 -> 结果变化”这个现象压缩成一段 CPU 演示。

rng = np.random.default_rng(42)
big = rng.normal(0, 1, size=200000).astype(np.float32)

order_a = big.copy()
order_b = big.copy()
rng.shuffle(order_b)

sum_a = np.sum(order_a, dtype=np.float32)
sum_b = np.sum(order_b, dtype=np.float32)

print('sum in original order =', sum_a)
print('sum in shuffled order =', sum_b)
print('absolute difference =', float(abs(sum_a - sum_b)))

[19] ✓ 0.0s Python

***
sum in original order = -151.56598
sum in shuffled order = -151.56584
absolute difference = 0.0001373291015625
```

作业

题目一：编写全局确定性引擎 (The Bulletproof Seed Engine)

在现代 CV 训练中，简单的 `torch.manual_seed()` 无法阻断底层的随机性泄漏。请利用 Vibe Coding 辅助，在 `utils.py` 中编写一个无死角的 `set_seed_everything(seed: int)` 函数。

开发约束：

1. 必须一次性锁定 Python 原生 `random`、`numpy.random` 以及 `torch`（涵盖 CPU 与所有可用 GPU）的随机状态。
2. 必须在代码中显式配置 `cuda.deterministic = True` 与 `cuda.benchmark = False`。
3. 必须通过 Python `os` 模块注入环境变量 `CUBLAS_WORKSPACE_CONFIG=:4096:8` 以应对底层 CUDA 算子的不确定性。
4. **核心要求：**编写一个可复用的 `worker_init_fn` 函数，用于挂载到 `DataLoader` 上。该函数必须能根据传入的 `worker_id` 动态为子进程生成独立且确定的随机种子（如 `base_seed + worker_id`）。

交付物：

包含上述所有逻辑的 Python 函数源码。

题目三：多进程 DataLoader 的“幽灵克隆”排查

场景设定：现有一段包含严重漏洞的数据加载代码。该代码使用了 `num_workers=4`，在自定义的 `Dataset __getitem__` 中调用了 `numpy.random.uniform` 来生成图像旋转角度和裁剪偏移量，且故意没有挂载 `worker_init_fn`。

行动指令：

1. **重现灾难：**编写代码实例化该 `DataLoader`，连续抽取前 16 张图片，并打印出每张图片实际应用的数据增强参数（如旋转角度）。
2. **现象分析：**观察控制台输出，找出不同 Worker 进程输出参数“完全克隆重复”的铁证。
3. **外科手术修复：**利用你写好的 Prompt 让大模型修复该 `DataLoader`，将你在【题目一】中写的 `worker_init_fn` 注入进去。

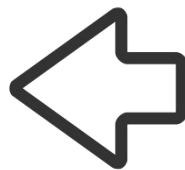
交付物：

修复前（包含重复参数的终端打印）与修复后（参数真正随机）的终端截图对比，以及修复后的 `DataLoader` 实例化代码。

模块一：从机器学习到深度学习

高维张量拓扑、连续内存布局与 Einops 重构

主讲人：利群他爹



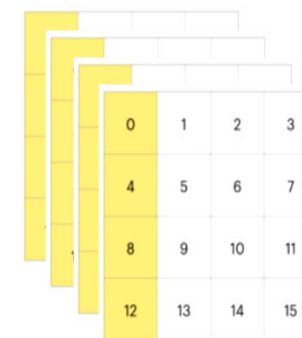
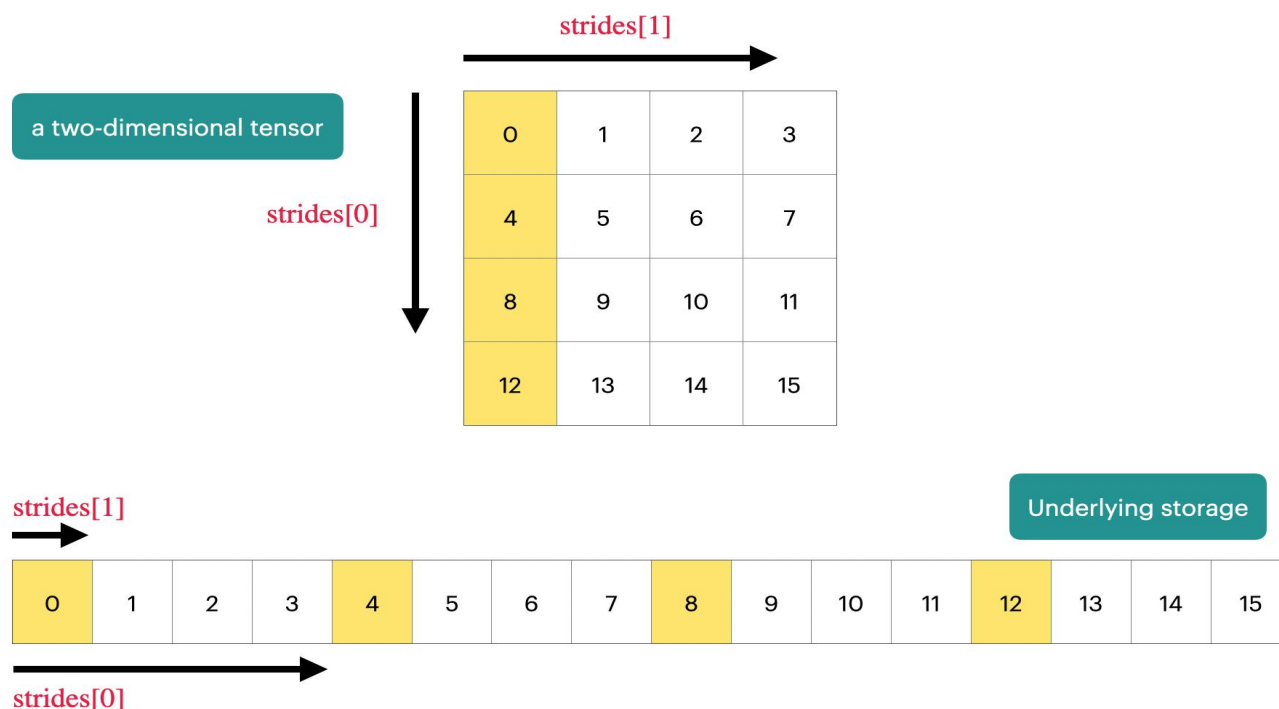
这是利群



我电脑里的张量



- 在物理内存（RAM / VRAM）中，张量本质上是一段连续的一维数组；所谓多维结构，只是解释这段内存的元数据视角。



那我拓展到三维，实际上就是二维的叠加，
strides[0]=16
也就是shape=a*b*c*...
strides[x]=...,a*b,a,1

那么好，对于一个4*4*4的张量
strides[2]=1;strides[1]=4,strides[0]=16

则有寻址公式：

$$Offset = \sum_{k=0}^{N-1} (i_k \times stride_k)$$

试求【2,1,3】位置

contiguous()连续及好处

- 一个张量是contiguous的，当且仅当它的内存排列正好是按字典序逐层展开的。即满足下式：

```
stride[N-1] = 1
stride[i] = stride[i+1] * shape[i+1]    (对所有的 i 都成立)
```

则这玩意连续



比如三维数组shape= (3,4,5),stride=(20,5,1):

最后维stride[2]=1

i=1:stride[1]应该等stride[2]xshape[2]=1x5=5

i=0:stride[0]应该等于stride[1]xshape[1]=5x4=20

当我存储的张量连续时，我的读写将极大节省时间（以显存换时间）

操作	显存	速度	使用场景
<code>permute</code> / <code>transpose</code>	✅ 零拷贝，不占额外显存	❌ 后续运算慢（跳跃读取）	临时调整布局，很快就 view 回去
<code>.contiguous()</code>	❌ 新开一块显存（翻倍）	✅ 后续运算全速	布局永久改变，接下来是密集计算
<code>reshape</code> 的隐式拷贝	❌ 悄悄占显存	⚠️ 不可控，调试地狱	最危险，不知道什么时候触发

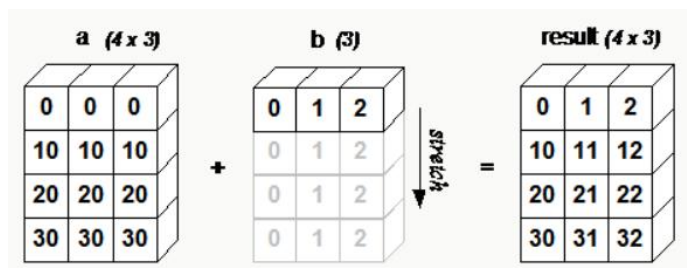
Contiguous、Zero-Copy 与 Cache Miss 的代价

- 你们可能不懂，我们在蹭别人算力的时候才知道的油盐贵：
- 以显存换时间（零拷贝）：不占显存，一看训练时长两万小时
- 以时间换显存（新开一块显存（翻倍））：兄弟你见过OOM吗
- 所以正确做法应该是：
 1. 在关键计算（如卷积、矩阵乘法、送入 Transformer block）之前，显式地确保张量是 contiguous 的，并且接受这次拷贝的显存开销。然后在该阶段密集运算中享受全速。
 2. 在只需要"换个角度看数据"的临时阶段（如调试、可视化、维度重排后马上又 permute 回去），保持零拷贝，别没事 contiguous。

聪明的朋友们，这一part过于底层了，我放了补充材料

幽默广播机制

- 幽默静步男的大，是发出和自己相反的波来抵消自己造出来的逼动静
- 幽默numpy的广播机制则是从右向左对齐维度，维度相等或一方为 1 即可广播
- 幽默pytorch在CPU中机制和原理保持完全一致（使用 stride = 0 的技巧。被广播的维度的 stride 直接设为 0，这意味着无论索引怎么变化，内存偏移都不变，永远指向同一个元素。
- 在GPU上则稍显复杂（聪明的朋友们自行理解）



python

复制 下载

```
import numpy as np
import torch

# NumPy
a_np = np.random.randn(2, 3, 4, 5) # (B, C, H, W)
b_np = np.random.randn(1, 3, 1, 1) # (1, C, 1, 1)
c_np = a_np + b_np # b_np 被广播, 底层不复制

# PyTorch
a_torch = torch.randn(2, 3, 4, 5)
b_torch = torch.randn(1, 3, 1, 1)
c_torch = a_torch + b_torch # 同上
```

Einstein Summation: 张量收缩的极致优雅代数表达

矩阵乘法 `C = A @ B` 用 einsum 写:

python

```
C = torch.einsum('ik, kj -> ij', A, B)
```

解读: `A` 的维度标记为 `i,k`, `B` 的维度标记为 `k,j`, 结果维度为 `i,j`。
重复出现的下标 `k` 表示在这个维度上**求和** (点积)。

多头自注意力分数:

python

```
attn = torch.einsum('bhid, bhjd -> bhij', Q, K)
```

- `b`: batch, `h`: head, `i,j`: 序列位置, `d`: 头维度。
- 对 `d` 求和, 生成 `i,j` 的分数矩阵。
- 一行代码干掉 `transpose` 和 `matmul` 的嵌套, 数学公式直接就是代码。

传统代码 (面条开始登场)

python

复制

```
batch, seq_len, d_model = Q.shape
num_heads = 8
d_k = d_model // num_heads

# 第一步: 拆头
Q = Q.reshape(batch, seq_len, num_heads, d_k)
K = K.reshape(batch, seq_len, num_heads, d_k)

# 第二步: 把 num_heads 移到第二维, 便于批量乘法
Q = Q.transpose(1, 2) # (batch, num_heads, seq_len, d_k)
K = K.transpose(1, 2) # (batch, num_heads, seq_len, d_k)

# 第三步: 计算注意力分数
# Q: (batch, num_heads, seq_len, d_k)
# K^T 需要把最后两维转置: (batch, num_heads, d_k, seq_len)
# 然后做批量矩阵乘法: (batch, num_heads, seq_len, d_k) x (batch, num_heads, d_k, seq_len)
K_T = K.transpose(-1, -2) # (batch, num_heads, d_k, seq_len)
attn_scores = torch.matmul(Q, K_T) # (batch, num_heads, seq_len, seq_len)
```

matmul的嵌套无异于西双版纳以南下暴雨——赤道湿了

作业

题目一：物理内存拓扑与 Stride 劫持验证

场景设定：深度学习中的张量底层仅是一维连续内存。你需要通过代码和数学推导，证明 `shape` 与 `stride` 只是这块一维内存的“虚拟解释滤镜”。

行动指令：

1. 实例化一个连续张量 $x \in \mathbb{R}^{B \times C \times H \times W}$ ，打印其内存首地址 (`.data_ptr()`)、`.shape`、`.stride()` 以及 `.is_contiguous()` 状态。
2. 执行 `y = x.permute(0, 2, 3, 1)`，再次打印 `y` 的上述 4 个属性。
3. 尝试直接对 `y` 执行 `.view()` 操作，记录并截图终端抛出的错误。
4. 使用 `.contiguous().view()` 修复该错误，并第三次打印操作后张量的内存首地址。
5. **理论验算：**在 Markdown 报告中，手画一维连续内存如何映射为二维张量，并给出一个具体的 Shape 和 Stride，手算演示元素在一维内存中的绝对偏移量公式。

交付物：

- 包含 3 次地址打印与报错截图的测试脚本。

作业

题目二：拒绝面条代码：Einops 空间变换重构

场景设定：传统 CV 代码中充斥着晦涩的 `.permute().contiguous().view()` 嵌套调用。你需要利用 Vibe Coding 辅助，将这些操作降维重构。

行动指令：

请使用 `einops.rearrange` 或 `einops.reduce` 完成以下 5 个计算机视觉核心场景的张量重构：

1. **特征展平：**将 (B, C, H, W) 展平为 $(B, H \times W, C)$ 。
2. **ViT Patch 切分：**将 (B, C, H, W) 切割为 $(B, N, P^2 \times C)$ 。
3. **多头注意力拆头：**将 $(B, N, H \times D)$ 拆分为 (B, H, N, D) 。
4. **BEV 多视角融合：**将 $(B, \text{Views}, C, H, W)$ 合并展开为序列。
5. **区域池化：**使用 `einops.reduce` 重写一个精确的区域最大值池化。
6. **附加题 (Einstein Summation)：**编写一个 `torch.einsum` 版本的 Attention 计算或 Batch Matmul 示例。

开发约束：

- 每次重构必须在代码注释中写明：输入 Shape、输出 Shape、语义变化，以及该操作是否触发了真实内存拷贝。
- 张量维度命名必须具有强烈的物理/几何含义（如 `b c h w`、`views`、`patch_h`），严禁使用 `i j k l` 等无意义字母。

交付物：

一份包含上述 6 个案例的 `einops_rewrite_cases.py` 代码库。

题目三：外科手术级异常探针 (Forward/Backward Hook)

场景设定：在混合精度训练 (AMP) 或复杂的网络（如带有跳跃连接的分割架构）中，梯度爆炸或特征图尺寸错位是家常便饭。你需要开发一个系统级探针来自动捕获它们。

行动指令：

1. **编写一个可插拔的 `ShapeNaNMonitor` 类。**该类利用 PyTorch 的 Hook 机制，自动监听挂载网络层的：
 - Forward 输出张量的 Shape。
 - Backward 传回梯度的 Shape。
 - 张量内是否出现 `NaN` 或 `Inf`。
 - 张量的 `is_contiguous` 状态。
2. **捕获演习：**编写一个简单的 MLP 网络，并故意在其中注入异常数值（如除以零）或梯度爆炸陷阱。将探针挂载到该网络上。
3. **熔断机制：**探针一旦在训练循环中检测到异常，必须立即中断训练，并在终端标红打印出“案发现场”的具体 Layer 名称和张量状态。

交付物：

- `shape_nan_monitor.py` 源码及成功熔断并打印出异常网络层的终端截图。
- 在代码头部写一段注释，解释为什么 Hook 是复杂网络 Debug 的终极武器。

模块一：从机器学习到深度学习

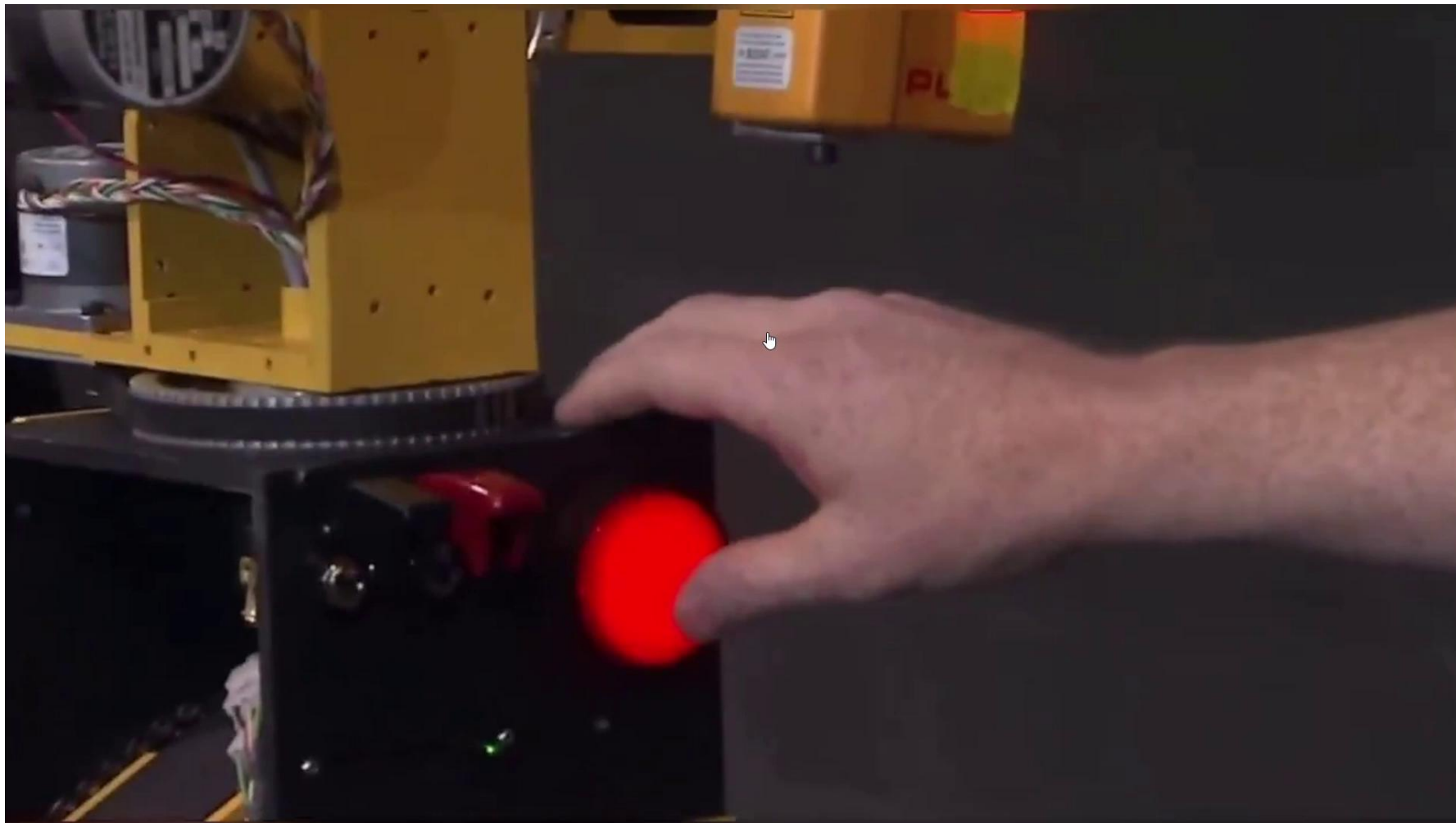
高性能数据管道与学术级统计可视化

主讲人：利群他爹

这是利群



为啥要用GPU



并行计算足够暴力
足够sexy

PCIe 带宽榨取：pin_memory 的物理本质

- `non_blocking=True` 的意义在于 CPU 可以立即返回继续准备下一个 batch，而 GPU 在后台通过 DMA 控制器异步拉取数据。但如果没有 `pin_memory`，CPU 必须先做同步的内存拷贝到锁页区域，这就破坏了异步性。

让我用一个具体的内存搬运路径图来帮你建立直觉：

text

【Pageable Memory (常规内存)】

CPU RAM → 临时锁页缓冲区 → PCIe 总线 → GPU

↑

可能被换出到磁盘

↑

两次拷贝，CPU 参与搬运

【Pinned Memory (锁页内存)】

CPU RAM (锁定) → PCIe 总线 → GPU (DMA 直接读取)

↑

单次拷贝，CPU 解放

常规可分页内存 (Pageable Memory) 的问题在于：

1. 操作系统随时可能把页面换出到磁盘。
2. 当 GPU 需要通过 PCIe 总线拉取数据时，如果源数据不在锁定内存中，CPU 必须先作为中转，把数据搬到一块临时锁页区域。
3. 这个过程会打断理想中的“异步数据拷贝”，训练线程很容易在等待搬运时被阻塞。

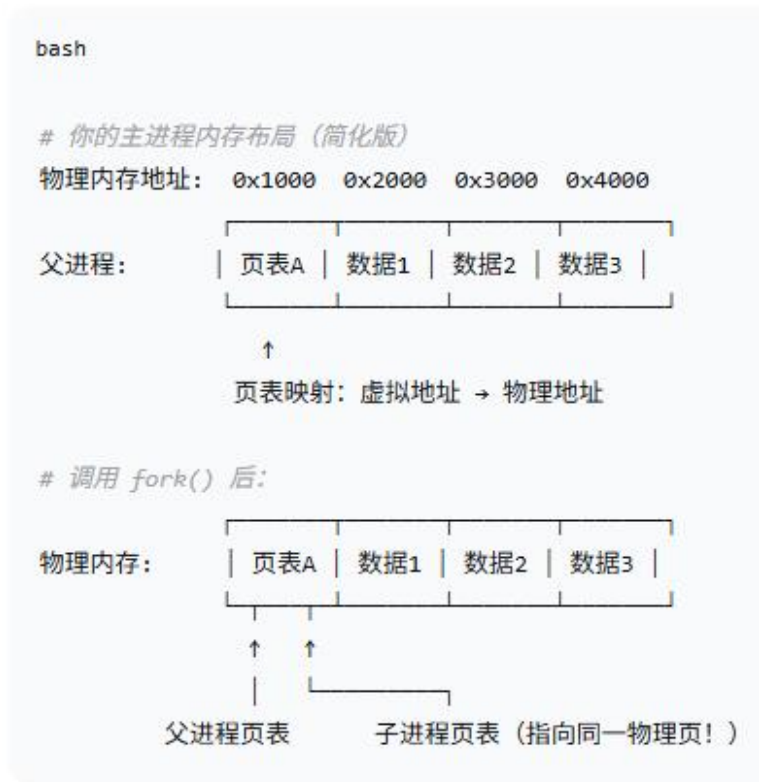
而锁页内存 (Pinned / Page-locked Memory) 的意义在于：

1. 这块内存被操作系统承诺不会被换出。
2. GPU 可以通过 DMA (Direct Memory Access) 控制器直接沿 PCIe 总线读取该区域。
3. 只有在这种前提下，`non_blocking=True` 才真正具有工程意义。

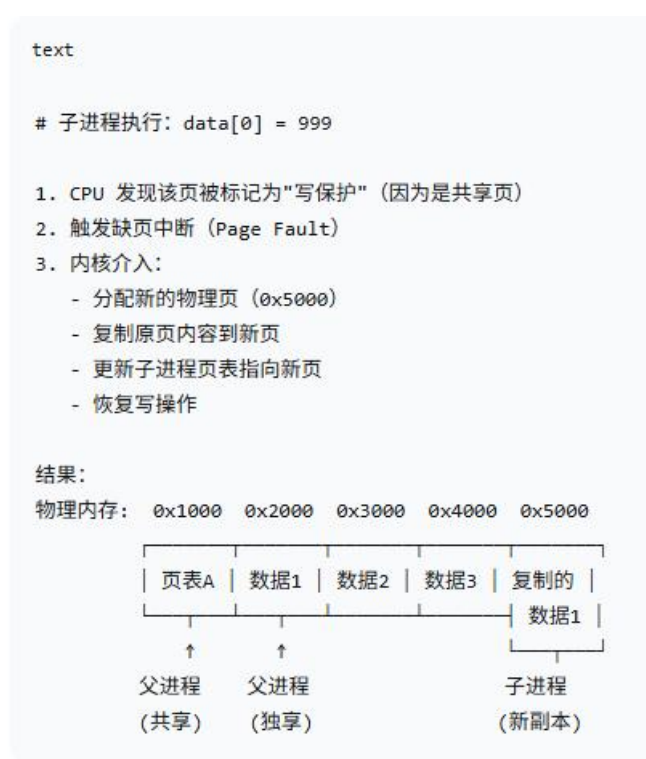
性能优化的本质是减少 CPU 充当搬运工的时间。

多进程与 CoW (Copy-on-Write) 内存炸弹

- 看看叉子fork():



- Copy-on-Write写时复制:



这就是"写时复制": 不写不复制, 一写就爆炸。

为啥fulljoy喷Python

Python 对象的真实结构

```
python
```

```
# 你写的代码
```

```
huge_list = [1, 2, 3]
```

```
# Python 解释器在内存中实际存储的东西:
```

PyObject_HEAD (引用计数、类型指针)	← 这部分会频繁变化!
长度、容量等元数据	
指向实际数据的指针数组	
实际数据: [1, 2, 3] 的 PyObject	← 每个元素也是对象

"读"操作为什么也会触发 CoW?

```
python
```

```
# 子进程中看似只读的操作
```

```
item = huge_list[0] # 你觉得是"读取"
```

实际上 Python 内部做了这些事:

```
python
```

```
# Python 解释器的真实执行流程
```

```
def fake_getitem(self, idx):
```

```
    # 1. 增加 self 的引用计数 (防止被 GC 回收)
```

```
    self->ob_refcnt += 1 # ← 这是写入操作! 触发 CoW!
```

```
    # 2. 从内部数组读取指针
```

```
    item_ptr = self->ob_item[idx]
```

```
    # 3. 增加返回值的引用计数
```

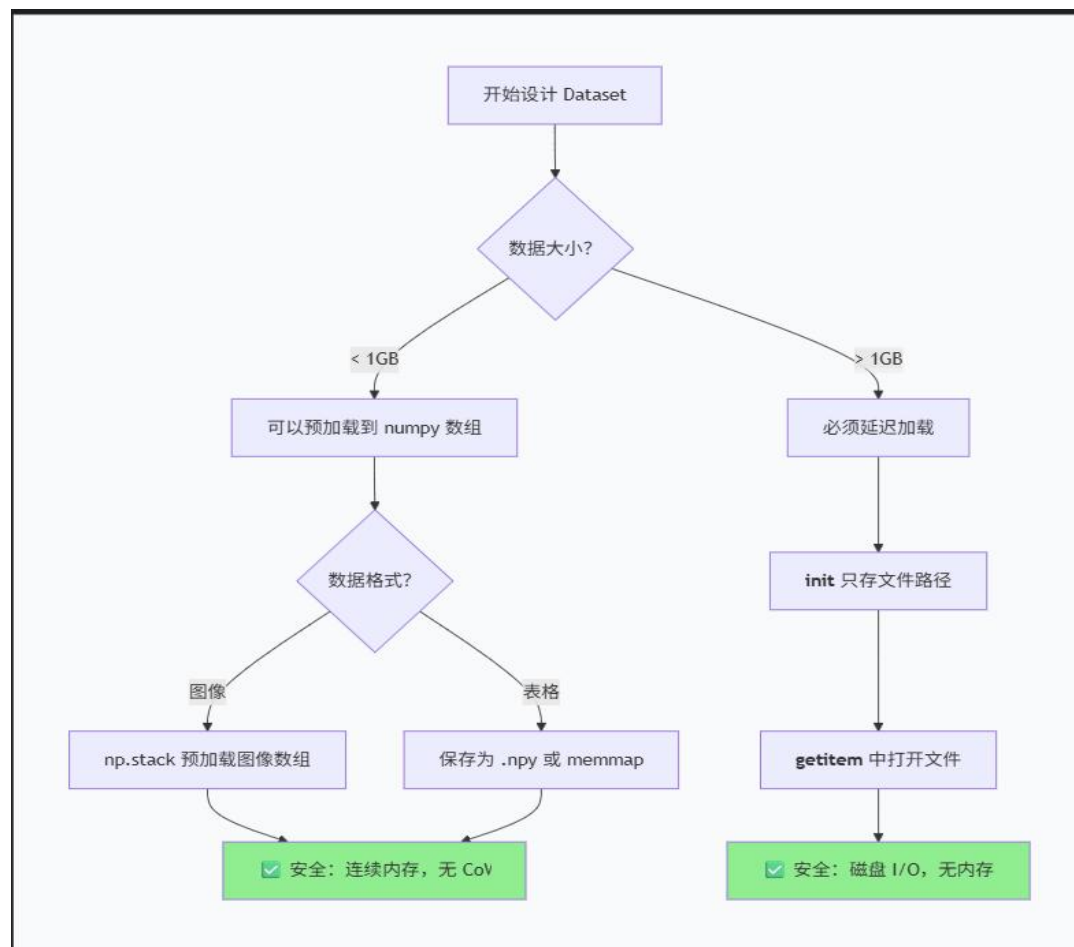
```
    item_ptr->ob_refcnt += 1 # ← 又是写入操作! 触发 CoW!
```

```
    return item_ptr
```

这就是为什么连"读取"都会爆炸: Python 的引用计数机制是一种隐式的写入操作。

优雅地做Dataset

- 多用零拷贝，多用np数组（这个你们可以自己查为啥可规避cow），init只放元数据



看代码

从 Welford 到并行方差聚合 (Chan's Method)

- welford当时在我面百度的时候被问到了（我也不知道这种他妈的小trick为啥要问）
 - welford用来求方差，我最直接的方差公式可用两行代码完成：
 - $\text{mean} = \text{sum}(\text{data}) / \text{len}(\text{data})$
 - $\text{variance} = \text{sum}((x - \text{mean})^2 \text{ for } x \text{ in data}) / (\text{len}(\text{data}) - 1)$
1. ❌ 需要两遍扫描数据
 2. ❌ 必须把所有数据存在内存里
 3. ❌ 对于流式数据（实时传感器、网络包）完全无法处理

所以就有了welford算法：可以只扫描一次数据，不需要存储历史数据，数值稳定性好。
下一页开始我们来手算一下：

代入 $\mu_{n+1} - \mu_n = \frac{x_{n+1} - \mu_n}{n+1}$, 化简即得:
 $M_{2,n+1} = M_{2,n} + (x_{n+1} - \mu_n)(x_{n+1} - \mu_{n+1})$

Welford 算法推导

已知:

- n 个样本后的均值: μ_n
- $M_{2,n} = \sum_{i=1}^n (x_i - \mu_n)^2$



第一步: 均值更新

$$\mu_{n+1} = \frac{n\mu_n + x_{n+1}}{n+1} = \mu_n + \frac{x_{n+1} - \mu_n}{n+1}$$

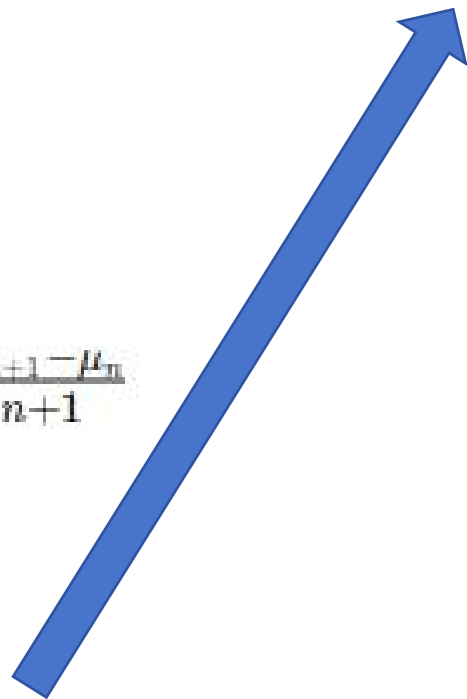


第二步: M2 更新 (这是魔法所在)

我们要证明:

$$M_{2,n+1} = M_{2,n} + (x_{n+1} - \mu_n)(x_{n+1} - \mu_{n+1})$$

看你妈呢, 自己推



```
"""
Welford 在线算法: 计算均值和方差
只需要 O(1) 内存, 不管数据有多少!
"""

count = 0      # 样本计数
mean = 0.0     # 当前均值
M2 = 0.0       # 平方差累积量 (Sum of squared differences)

for x in data_stream:
    count += 1
    delta = x - mean          # 新值与旧均值的差
    mean = mean + delta / count # 更新均值
    delta2 = x - mean         # 新值与新均值的差
    M2 = M2 + delta * delta2  # 累积平方差
```

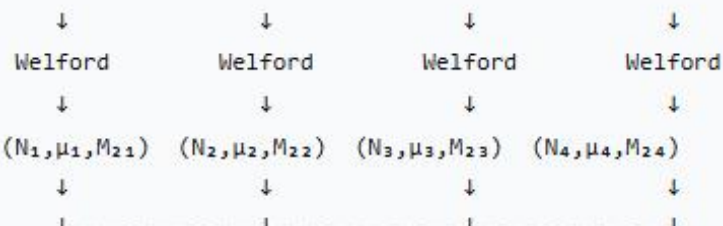
Chan's Method

我们现在算完了welford（串行），面对并行的welford则需要进行方差聚合

text

你的数据：

第1块	第2块	第3块	第4块
25万张图	25万张图	25万张图	25万张图



如何合并成全局统计量？

Chan's Method!

块 A:

- 样本数: N_A (比如 100 个样本)
- 均值: μ_A (比如 5.0)
- 平方差累积量: $M_{2,A} = \sum_{i \in A} (x_i - \mu_A)^2$

块 B:

- 样本数: N_B (比如 200 个样本)
- 均值: μ_B (比如 8.0)
- 平方差累积量: $M_{2,B} = \sum_{j \in B} (x_j - \mu_B)^2$

只用这 6 个数字，计算出合并后的：

$$M_{2,A \cup B} = \sum_{x \in A \cup B} (x - \mu_{A \cup B})^2$$

其中 $\mu_{A \cup B}$ 是合并后的全局均值。

$$M_{2,A \cup B} = M_{2,A} + M_{2,B} + \frac{N_A N_B}{N_A + N_B} (\mu_A - \mu_B)^2$$

Chan's Method的推广

- 如次美妙的修正项：

$$M_{2,A \cup B} = M_{2,A} + M_{2,B} + \frac{N_A N_B}{N_A + N_B} (\mu_A - \mu_B)^2$$

- 看到这个鸡在河上飞的幽默形式，不禁令人想起小学二年级的并联电阻求电阻和
- 智力水平略逊于二年级的各位，不妨想想三项及三项以上的方差聚合的推导怎么样？
- 做不出来的话，欧姆会在天上失望的看着你们
- 推完三项，那不就可以归纳出了k项的形式吗

Hierarchical defect-rich flower-like BiOBr/Ag nanoparticles/ultrathin g-C₃N₄ with transfer channels plasmonic Z-scheme heterojunction photocatalyst for accelerated visible-light-driven photothermal-photocatalytic oxytetracycline degradation

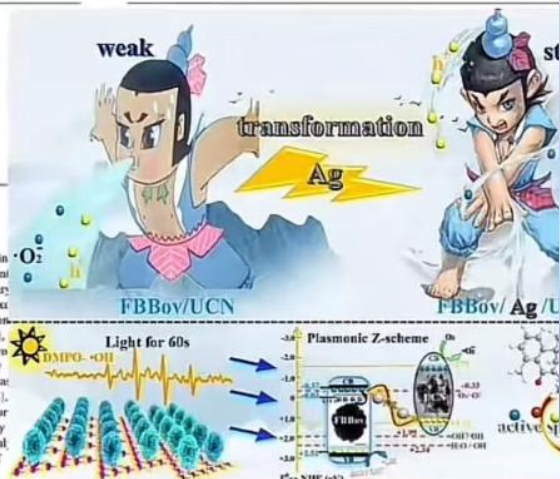
Jiaxin Ni^a, Dongmei Liu^{a,*}, Wei Wang^a, Aiwen Wang^a, Jialin Jia^a, Jiayu Tian^c, Zipeng Xing^{b,*}

^a State Key Laboratory of Urban Water Resource and Environment (SKLURWE), School of Environment, Harbin Institute of Technology, Harbin, 150001, PR China
^b Department of Environmental Science, School of Chemistry and Materials Science, Key Laboratory of Functional Porous Material Chemistry, Ministry of Education of the People's Republic of China, Heilongjiang University, Harbin 150008, PR China
^c School of Civil Engineering and Transportation, Hebei University of Technology, Tianjin 300401, China

ARTICLE INFO

Keywords:
Oxytetracycline
Plasmonic Z-scheme
Surface oxygen vacancies
Visible-light photocatalysis
Photothermal effect

ABSTRACT



1. Introduction

Antibiotics, as a kind of medicines, have been constantly used in the pharmaceutical industry of its low price and excellent efficacy [1]. Therefore, more attention has been paid to the development of antibiotics. The photocatalytic degradation of antibiotics is regarded as an effective method [4,5]. Antibiotics are the most important catalysts to effectively various new photocatalysts. BiOBr, a kind of

Enhanced Nanobubble Formation: Gold Nanoparticle Conjugation to Q β Virus-like Particles

Perouza Parsamian, Yaning Liu, Chen Xie, Zhuo Chen, Peiyuan Kang, Yalini H. Wijesundara, Noora M. Al-Kharji, Ryanne Nicole Ehrman, Oriyeda Trashi, Jaona Randrianalisoa, Xiangyu Zhu, Matthew D'Souza, Lucas Anderson Wilson, Moon J. Kim, Zhenpeng Qin,^{*} and Jeremiah J. Gassensmith^{*}

Cite This: <https://doi.org/10.1021/acs.nano.3c00438>

Read Online

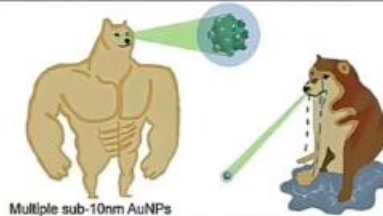
ACCESS |

Metrics & More

Article Recommendations

Supporting Information

ABSTRACT: Plasmonic gold nanobubbles (AuNBs) are used in modern hypersensitive analyt photoacoustic, bioimaging, and biosensing. However, the plasmonic nanobubble cavitation event is stochastic, specifically including small nanobubbles (sub-10 nm) which lack size control, tunability by use of ultrashort pulses (ns, ps) and can result in tissue and cellular damage. This work investigates a method to immobilize and size control (5 nm) onto a chemically modified virus-like particles. These findings demonstrate a profound and disproportionate increase in the number of AuNBs.



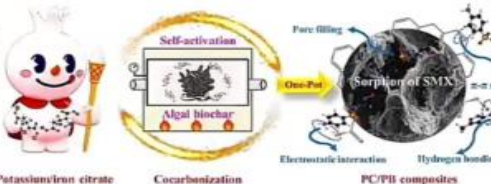
Self-activation of potassium/iron citrate-assisted production of porous carbon/porous biochar composites from macroalgae for high-performance sorption of sulfamethoxazole

Jiaohong Qin^a, Rongting Ji^b, Qian Sun^{a,c}, Wei Li^{a,d}, Hu Cheng^{a,e,f}, Jiangang Han^{a,d}, Xin Jiang^a, Yang Song^a, Jianming Xue^{a,*}

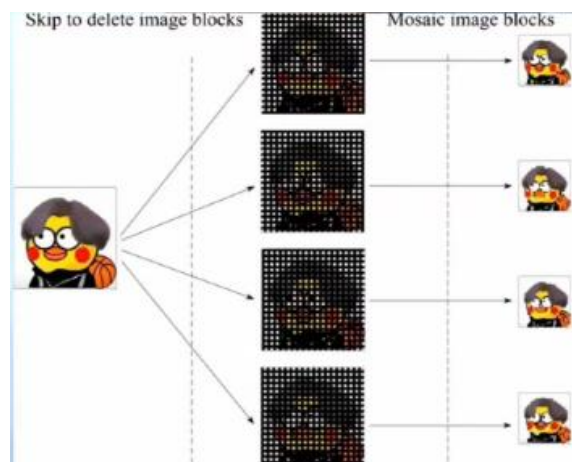
^a Co-Innovation Center for the Sustainable Forestry in Southern China, College of Biology and the Environment, Nanjing Forestry University, Nanjing 210047, PR China
^b Nanjing Institute of Environmental Sciences, Ministry of Ecology and Environment, Nanjing 210042, PR China
^c College of Agricultural Science and Engineering, Anhui University, Hefei 230026, PR China
^d National Forestry Engineering Technology Research Center of Macroeconomic, Beijing 100080, PR China
^e CAS Key Laboratory of Soil Environment and Pollution Remediation, Institute of Soil Science, Chinese Academy of Sciences, Nanjing 210009, PR China
^f New Zealand Forest Research Institute Limited, Christchurch 8000, New Zealand

HIGHLIGHTS

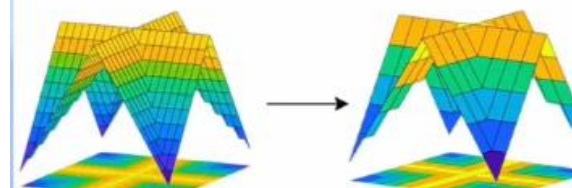
- PC/BC composites developed with the help of the self-activation process.
- PC/BC composites developed by the PC/BC.
- PC/BC composites developed by the PC/BC.
- PC/BC composites developed by the PC/BC.
- PC/BC composites developed by the PC/BC.



], 我也不知道是不是我审美问题, 我求求了



Apply jumping strategy to higher-dimensional models



科研绘图

- 我就放作业里了，你们vibcoding的时候感受吧

题目一：MapReduce 并行图像统计引擎

场景设定：面对 ImageNet 级别的海量数据，单线程串行统计均值与方差会耗费大量时间。你需要实现一个高并发的流式统计引擎。

行动指令：

1. 编写多进程脚本（利用 `multiprocessing` 或多 Worker DataLoader），将数据分块交由多个 Worker 独立处理。
2. 每个 Worker 独立计算并输出其分块数据的：样本数 N 、均值 μ 、以及平方差累积量 M_2 。
3. 主进程收集所有 Worker 的结果，并严格使用 Chan's Method (并行方差合并公式) 在 $O(1)$ 时间内聚合出全局的最终均值与标准差。
4. **理论推导：**在报告中手推出 Chan 公式中的偏移量惩罚项 $\frac{N_A N_B}{N_A + N_B} (\mu_A - \mu_B)^2$ ，并用一句话解释其统计学意义。

交付物：

包含分块计算与合并逻辑的高并发统计 Python 脚本，以及公式手推过程。

题目二：排查 DataLoader 多进程内存泄漏 (CoW 陷阱)

场景设定：教师在 `dataset_stats_and_plot.py` 中故意在 Dataset 的 `__init__()` 里留下了一个巨大的嵌套 Python 字典（“毒药”数据结构）。

行动指令：

1. **重现灾难：**将 DataLoader 的 `num_workers` 设为 8 并启动。打开系统资源监控器 (Task Manager / `htop`)，记录并截图内存呈阶梯式暴涨直至 OOM 的过程。
 2. **理论分析：**解释为什么 Linux `fork()` 下的 Copy-on-Write (CoW) 机制会让 Python 的高层对象在多进程中变成“内存炸弹”。解释为何开启 `pin_memory=True` 必须配合 `non_blocking=True` 才能实现真正的异步传输。
 3. **外科手术修复：**利用 Vibe Coding 辅助，重构 Dataset 的底层数据结构。将其替换为连续内存格式（如 `numpy.memmap`、连续 NumPy 数组或采用延迟读取策略）。
 4. **验证：**再次开启 8 个 Worker 运行，提供主内存占用趋于平稳、无暴涨现象的监控截图。
- 交付物：**

内存泄漏前后的系统监控截图对比、问题原因的理论解释，以及修复后的 Dataset 源码。

题目三：开发 IEEE 顶刊级矢量图引擎

场景设定：告别随手写的 `plt.figure()`，你需要建立一个系统级的、符合学术顶刊标准的可视化引擎。

行动指令：

1. 封装一个统一的绘图引擎类（通过修改 `matplotlib.rcParams` 或等价方式配置全局样式）。
2. 强制约束：
 - 禁止输出光栅图，强制保存为 `.pdf` 或 `.svg`。
 - 去除顶部和右侧的边框 (Spines)。
 - 配置正确的字体族，确保中英文混排绝不乱码。
 - 图例 (Legend) 必须无边框，配色强制使用色盲友好色板。
3. 成果渲染：调用该引擎，输出两张图表：(1) 数据分布统计图；(2) 数据增强前后的样本对比图。将它们组织成一页 PDF 画板。

交付物：

- 封装完整的绘图引擎源码。
- 一份导出的 PDF 画板（验收标准：在 PDF 阅读器中放大 500% 后，坐标轴文字和曲线边缘必须绝对锐利、无任何锯齿或马赛克）。

第一模块结束

- 那么好，我们终于完成了最简单最水的part
- 来享受真正的折磨和痛苦吧
- 喵
- 喵喵喵
- 喵喵喵喵喵
- 桀桀桀桀桀桀桀